

Deep Learning Avancé & Applications

Fine-tuning, PEFT & RAG

Master Bac+5 – Data Analytics & Big Data
Mouaad AGOURRAM

2025–2026

Golden Collar

Fine-tuning des LLMs

LoRA & ses Familles — Adaptation Efficace

RAG — Retrieval-Augmented Generation

Les Familles de RAG

Comparaison — LoRA vs RAG

Data Flywheel : Boucle RAG → LoRA

Travaux Pratiques

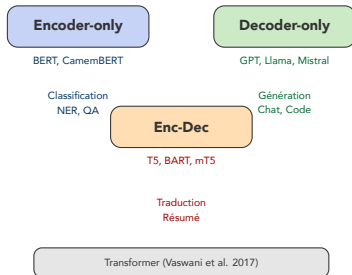
Fine-tuning des LLMs

Les 3 familles de Transformers

- **Encoder-only** (BERT, CamemBERT) : bidirectionnel, compréhension
- **Decoder-only** (GPT, Llama, Mistral) : auto-régressif, génération
- **Enc-Dec** (T5, BART) : traduction, résumé

L'explosion des tailles

- 2018 : BERT = 110M params (1 GPU suffit)
- 2020 : GPT-3 = 175B params
- 2024 : Llama 3 = 8B, 70B, 405B params
- **Scaling Laws** (Kaplan, 2020) : la performance suit une loi de puissance prévisible en fonction de la taille, des données et du compute

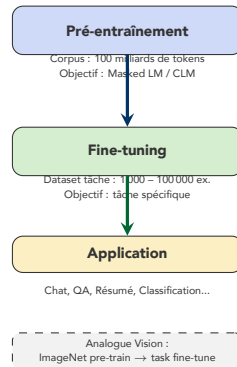


Le Paradigme Pre-train → Fine-tune

Le changement de paradigme (2018+)

1. **Pré-entraînement** sur corpus massif (Wikipedia, CommonCrawl)
— objectif auto-supervisé (Masked LM, CLM)
2. **Fine-tuning** sur tâche spécifique avec peu de données labellisées
3. **Résultat** : transfert de connaissance, SOTA avec $1\,000\times$ moins de données

Le modèle pré-entraîné encode les **structures du langage** ;
le fine-tuning le *spécialise*.



Principe

- **Tous** les paramètres du LLM sont mis à jour
- Learning rate différentiel : couches basses $\approx 10^{-5}$, hautes $\approx 10^{-4}$
- Meilleure performance quand les données abondent ($> 50k$ exemples)
- Utilisé pour changer complètement le style/domaine du modèle

Le coût prohibitif sur les LLMs

Pour un modèle de 8B paramètres en bf16 :

- Poids (16 GB) + gradients (16 GB) + Adam (32 GB) $\approx$ **64 GB VRAM**
- Sauvegarde : **16 GB par tâche** (une copie complète)
- Risque de *catastrophic forgetting* élevé
- Multi-tâche = N copies du modèle

⇒ **Impraticable** pour la majorité des cas d'usage industriels.

Supervised Fine-Tuning (SFT)

- Dataset de paires (*instruction*, *réponse attendue*)
- Le modèle apprend à **suivre des consignes**
- Formats : Alpaca, ShareGPT, ChatML
- Exemple : "Résume ce texte en 3 points" → réponse structurée

RLHF / DPO — Alignement par préférences

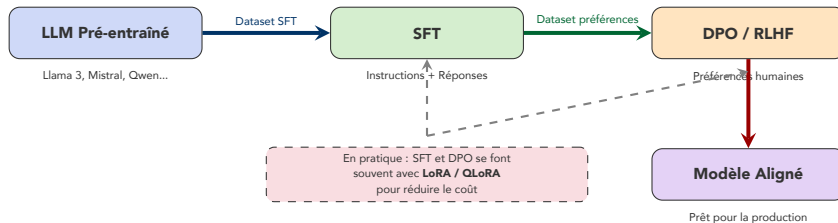
- **RLHF** : Reward Model + PPO — le modèle maximise une récompense apprise sur les préférences humaines
- **DPO** (Direct Preference Optimization) : pas besoin de RM, optimisation directe sur les paires ($y_w \succ y_l$)

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)$$

π_{ref} = modèle de référence (gelé). β contrôle l'écart maximal autorisé : le ratio pénalise les dérives trop importantes.

- **GRPO** (DeepSeek) : Group Relative Policy Optimization — sans critic model

Pipeline complet de Fine-tuning d'un LLM



Les 3 étapes du fine-tuning moderne

1. **Continual Pre-training** (optionnel) : adapter le vocabulaire à un domaine (médical, juridique)
2. **SFT** : apprendre à suivre des instructions avec des paires (prompt, réponse)
3. **Alignement** (DPO/RLHF) : affiner le comportement via les préférences humaines

LoRA & ses Familles — Adaptation Efficace

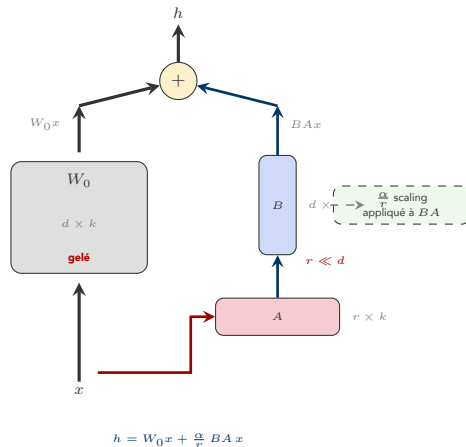
Hypothèse clé — Rang intrinsèque bas

Motivation : Aghajanyan et al. (2020) montrent que le fine-tuning opère dans un sous-espace de faible dimension intrinsèque.

Hu et al. (2021, LoRA) exploitent cela : ΔW peut être approché par un produit de **rang faible**.

$$h = W_0 x + \frac{\alpha}{r} (BA) x$$

$W_0 \in \mathbb{R}^{d \times k}$ **gelé** ; seuls $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$ s'entraînent, $r \ll \min(d, k)$.



L'initialisation — élément crucial

- W_0 (modèle pré-entraîné) : **gelé**, aucun gradient calculé.
- $A \sim \mathcal{N}(0, \sigma^2)$: initialisation aléatoire.
- $B = 0$: initialisation à zéro. **Indispensable** : si $B \neq 0$, alors $\Delta W = BA \neq 0$ dès le départ $\Rightarrow$ le modèle est perturbé et l'entraînement devient instable.
- **Au départ** : $BA = 0 \Rightarrow$ le modèle se comporte *exactement* comme le modèle de base. L'adaptation émerge progressivement.
- α : facteur de scaling ; empiriquement $\alpha = r$ ou $2r$.

Gain d'efficacité — Exemple concret

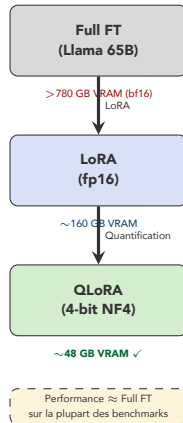
Si $d = k = 4096$ et $r = 8$:

- ΔW entier = $4096^2 = 16.7\text{M}$ params.
- LoRA ($A + B$) = $2 \times 4096 \times 8 = 65\,536$ params.
- **Réduction de 99.6%** des poids entraînaables.
- En pratique, les adapters LoRA font quelques **mégaoctets** vs 16 GB pour le modèle complet.

QLoRA (Dettmers et al., 2023)

Quantized LoRA : la version qui a démocratisé les LLMs locaux.

- W_0 quantifié en **4-bit NormalFloat** (NF4) — format optimisé pour les distributions normales des poids
- Charge un modèle 70B (~ 140 GB) sur un seul GPU de 48 GB
- **Double quantification** : quantifie aussi les constantes de quantification
- **Paged Optimizers** : décharge l'état Adam sur CPU en cas de pic mémoire



DoRA — Weight-Decomposed LoRA (Liu et al., ICML 2024)

Sépare la matrice de poids en deux composantes :

$$W = m \cdot \frac{V}{\|V\|_c}$$

1. **Magnitude** m : vecteur d'échelle (appris directement)
2. **Direction** V : modifiée via LoRA standard
($V = W_0 + BA$)

Reproduit le comportement d'apprentissage du Full Fine-tuning bien mieux que LoRA classique.

Pourquoi ça marche mieux ?

- Le Full FT modifie **beaucoup** la direction et **peu** la magnitude
- LoRA classique couple les deux $\Rightarrow$ sous-optimal
- DoRA découple : la direction évolue librement via LoRA, la magnitude est apprise indépendamment
- **Gain notable** sur les tâches de raisonnement complexe
- Surcoût mémoire négligeable vs LoRA

Approches PEFT populaires

- **LoRA+** : learning rate asymétrique ($LR_B \gg LR_A$, typiquement $\times 16$). Gain de convergence notable sur certains benchmarks.
- **VeRA** : A et B aléatoires, gelées et partagées entre couches. Seuls de petits vecteurs de scaling sont entraînés par couche.
- **LoKr / LoHa** : Produit de Kronecker / Hadamard pour ΔW — encore moins de paramètres.
- **AdaLoRA** : alloue le rang r dynamiquement par couche selon l'importance (SVD).

En pratique — Librairie PEFT (HuggingFace)

- La librairie `peft` centralise toutes ces implémentations.
- `LoraConfig`, `get_peft_model()`, `SFTTrainer` : 10 lignes de code.
- **Hyperparamètres clés** : r (rang), α (scaling), `target_modules` (Q, K, V, FFN ?).
- Intégration native avec `transformers` et `trl`.

Méthode	Params entr.	VRAM (bf16)
Full FT	100%	$\sim 4\times$ poids
LoRA	0.1–1%	$\sim 1.2\times$ poids
QLoRA	0.1–1%	$\sim 0.5\times$ poids
VeRA	$<0.01\%$	$\sim 0.5\times$ poids

RAG — Retrieval-Augmented Generation

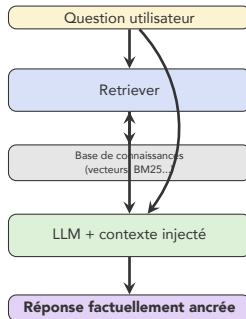
Le Problème des LLMs sans Mémoire Externe

Les 3 limites fondamentales des LLMs

1. **Hallucinations** : le modèle génère des faits plausibles mais faux avec une confiance totale
2. **Knowledge cutoff** : connaissances figées à la date d'entraînement
3. **Données privées** : impossible d'accéder aux documents internes

L'intuition RAG

"Donner des yeux au LLM" : chercher les informations pertinentes puis générer en s'appuyant dessus.
Analogie : un avocat qui consulte la jurisprudence avant de plaider.



Formulation originale — Lewis et al., NeurIPS 2020

$$p(y | x) = \sum_{z \in \mathcal{Z}} \underbrace{p_{\eta}(z | x)}_{\text{Retriever}} \cdot \underbrace{p_{\theta}(y | x, z)}_{\text{Générateur}}$$

- x : question de l'utilisateur
- $z \in \mathcal{Z}$: document retrouvé (passage)
- p_{η} : probabilité de retrouver z étant donné x
- p_{θ} : probabilité de générer y avec x et z
- En pratique : top- k chunks ($k \in \{3, 5, 10\}$) concaténés dans le prompt

Similarité cosinus et BM25

Dense (similarité cosinus) :

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} = \cos(\theta)$$

Sparse (BM25) :

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{|d|}\right)}$$

BM25 capture les termes exacts, noms propres, acronymes.

Pourquoi chunker ?

- Les modèles d'embedding ont une fenêtre limitée (512 tokens pour BERT)
- Embedder un document entier = perte de granularité sémantique
- Un chunk = l'unité de recherche et de contexte

Trade-off taille de chunk

- **Trop petit** (<100 tokens) : manque de contexte, retrieval peu précis
- **Trop grand** (>1000 tokens) : dilution du signal, LLM submergé
- **Sweet spot** : 200–500 tokens avec 10–20% overlap

Stratégies de chunking			
Fixed size	Sentence-based	Recursive	Parent-Child
Couper tous les N tokens. Simple, ignore la sémantique.	Couper aux fins de phrases. Respecte les unités linguistiques.	Couper sur \n\n, \n, ".", espace. Compromis structure + taille.	Chunks petits pour le retrieval, chunks larges envoyés au LLM.

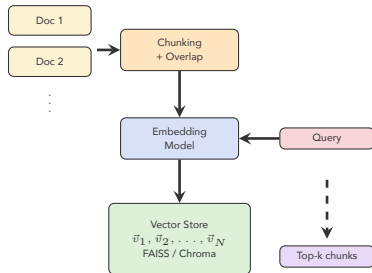
Pipeline d'Indexation — Embedding & Vector Store

Modèles d'embedding populaires

- **Sentence-BERT** : bi-encoder, optimisé pour la similarité sémantique
- **E5 / BGE** (Microsoft/BAAI) : parmi les meilleurs sur le benchmark MTEB
- **BAAI/bge-m3** : multilingue, 100+ langues dont le français
- **text-embedding-3-large** (OpenAI) : 3072 dims, très performant

Vector Stores

- **FAISS** (Meta) : open-source, in-memory, HNSW/IVF, ultra-rapide
- **ChromaDB** : simple, local ou cloud, idéal pour les TP
- **Weaviate / Qdrant** : production-grade, filtres metadata
- **Pinecone** : SaaS managé, scalable



Hybrid Search

BM25 rate les synonymes et paraphrases ; le dense retrieval rate les correspondances exactes (noms propres, acronymes).

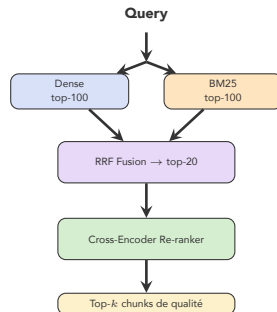
$$\text{score}_{\text{hybrid}}(d) = \alpha \cdot \text{score}_{\text{dense}}(d) + (1 - \alpha) \cdot \text{score}_{\text{BM25}}(d)$$

En pratique : **Reciprocal Rank Fusion (RRF)** :

$$\text{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + r(d)}, \quad k = 60$$

Re-ranking — bi-encoder vs cross-encoder

- **Bi-encoder** : q et d encodés séparément → rapide (retrieval)
- **Cross-encoder** : (q, d) encodés ensemble → précis (re-ranking)
- **Pipeline** : Bi-encoder top-100, Cross-encoder re-classe en top-10



Les Familles de RAG

Retrieve → Read → Generate

1. **Ingestion** : chunks + embeddings dans le vector store
2. **Retrieval** : similarité cosinus directe sur la question brute
3. **Génération** : prompt = question + top-k chunks concaténés

C'est la version de base — la plus simple à implémenter.

Limites du Naive RAG

- La qualité du retrieval est le **goulot d'étranglement absolu**
- Pas de réécriture de la requête — si l'utilisateur formule mal, le retrieval échoue
- Pas de vérification post-retrieval — chunks non pertinents passent
- **Lost in the middle** (Liu et al., 2023) : les LLMs accordent plus d'attention au début et à la fin du contexte qu'au milieu

Pré-retrieval : transformer la requête

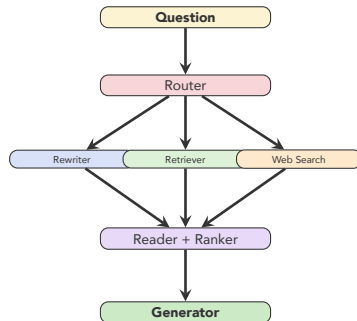
- **HyDE** : le LLM génère un document hypothétique, on embed ce document au lieu de la question
- **Multi-Query** : N reformulations $\rightarrow$ union dédoublée des résultats
- **Step-Back** : question plus abstraite pour retrouver les principes généraux
- **Query Routing** : diriger vers la bonne source

Post-retrieval : affiner le contexte

- **Re-ranking** : cross-encoder pour réordonner par pertinence
- **Compression** : résumer les chunks longs en conservant l'essentiel
- **Filtrage** : éliminer les chunks sous un seuil de pertinence
- **Reordering** : placer les chunks les plus pertinents en début et fin du prompt (éviter le *lost in the middle*)

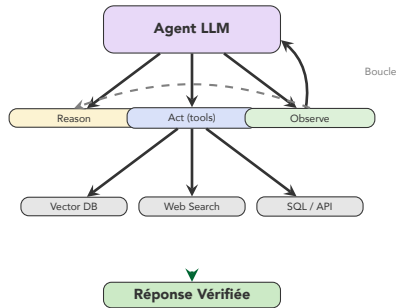
Principe

- Modules interchangeables : Retriever, Rewriter, Reader, Generator
- Routing dynamique selon la nature de la question (factuelle / analytique / créative)
- Chaque module peut être mis à jour indépendamment
- Flexibilité maximale pour les systèmes de production



Principe — Boucle ReAct

- L'agent **décide** : quand chercher, dans quelle source, si la réponse suffit
- Boucle **ReAct** : *Reason* → *Act* → *Observe* → *Repeat*
- Peut effectuer plusieurs passes de retrieval et de vérification
- **Tendance 2025** : SLMs locaux (Llama-3-8B) pour orchestrer à bas coût

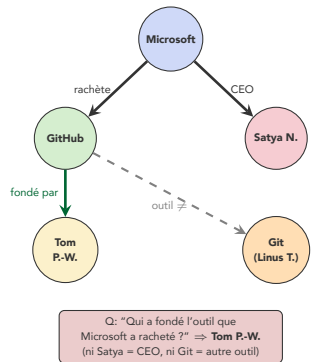


Limite du RAG vectoriel

- Retrieval par chunks : efficace pour les **faits locaux**
- Échoue sur les **questions multi-hop** : “Quel est le fondateur de l’entreprise qui a racheté GitHub ?”
- Les chunks pertinents sont dans des documents séparés, liés par des entités

Graph RAG (Microsoft Research, 2024)

1. **Extraction** : LLM extrait entités et relations → Knowledge Graph
2. **Indexation** : communautés de nœuds, résumés hiérarchiques
3. **Traversée** : parcours du graphe guidé par la question
4. **Génération** : chemin de raisonnement injecté comme contexte



Idée clé

Ces architectures **fusionnent** retrieval et génération : le LLM ne se contente plus de consommer les documents, il **pilote activement** le processus de recherche et **critique** ses propres résultats.

Self-RAG (Asai et al., 2023)

- Le LLM **décide lui-même** quand il a besoin de chercher
- Génère des **tokens de réflexion** spéciaux : [Retrieve], [IsRel], [IsSup], [IsUse]
- Critique et corrige ses propres réponses en boucle
- Évite les retrievals inutiles sur les questions simples

Corrective RAG (Yan et al., 2024)

- Évalue la qualité du retrieval avec un **grader** automatique
- Si le score est bas : **fallback** vers une recherche web
- Décompose les documents en bandes de connaissances (knowledge strips)
- Filtre les informations non pertinentes avant génération

≠ RAG classique : ici le LLM est dans la boucle de retrieval, pas juste consommateur.

4 métriques fondamentales (RAGAS)

- **Faithfulness** : la réponse est-elle ancrée dans le contexte ?

$$\text{Faith.} = \frac{|\text{Affirmations supportées}|}{|\text{Total affirmations générées}|}$$

- **Answer Relevancy** : la réponse adresse-t-elle la question ?
- **Context Precision** : les chunks utiles sont-ils bien classés en haut ?
- **Context Recall** : a-t-on retrouvé tous les éléments nécessaires ?

Problèmes courants et solutions

- **Chunks trop petits** : contexte insuffisant → Parent-Child indexing
- **"Lost in the middle"** : LLM ignore le contexte central → ordonner les chunks par pertinence décroissante
- **Mauvais retrieval** : espaces sémantiques différents → HyDE ou fine-tuning du retriever
- **Hallucination malgré RAG** : → prompt engineering + faithfulness check

Comparaison — LoRA vs RAG

Quand utiliser quoi ? — Critères de décision

Full Fine-tuning si...

- Données labellisées abondantes (> 50k)
- Changement de style ou de domaine complet
- Ressources GPU massives disponibles
- Données **statiques** (ne changent pas)

LoRA / QLoRA si...

- GPU limité → QLoRA (1 GPU 24 GB)
- Données de qualité disponibles (50 – 50k exemples)
- Plusieurs tâches sur 1 même modèle de base
- Adapter le style, la langue ou le format de sortie

RAG si...

- Connaissance **évolutive** (mises à jour fréquentes)
- Sources citables, traçabilité absolue requise
- Pas de données labellisées disponibles
- Données privées / confidentielles

Règle d'or

Épuiser d'abord **Prompt Engineering** → **RAG** → **LoRA** avant d'envisager le Full Fine-tuning.

Tableau Comparatif — Cheat Sheet

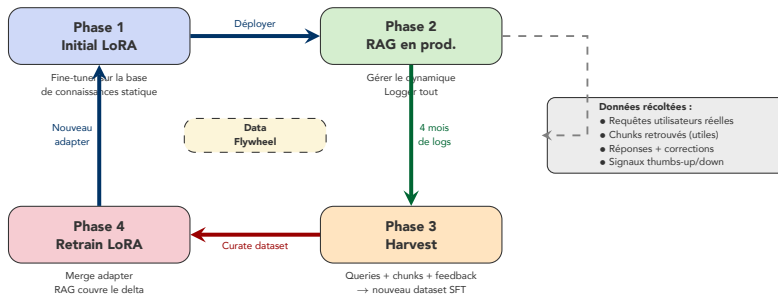
Critère	Full FT	LoRA	QLoRA	RAG
Paramètres modifiés	100%	0.1–1%	0.1–1%	0%
VRAM requise	~4× poids (bf16)	~1.2×	~0.5×	modèle seul
Données labellisées	Beaucoup	Modérées	Modérées	Aucune
Fraîcheur des données	Figée	Figée	Figée	Temps réel
Citabilité des sources	Non	Non	Non	Oui
Catastrophic forgetting	Risque élevé	Faible	Faible	Nul
Adaptation multi-tâche	1 modèle / tâche	Module swap	Module swap	1 base

En pratique — combiner les approches

LoRA + RAG = modèle adapté au domaine (style, vocabulaire) ET contexte dynamique au runtime.
Ex : LLM médical (QLoRA sur corpus clinique) + RAG sur dossiers patients (dynamique, confidentiel).

**Data Flywheel : Boucle RAG →
LoRA**

Le Cycle RAG → LoRA — Data Flywheel



Note : “Data Flywheel” est un **pattern d’ingénierie de production** (NVIDIA NVInfo, boucles MAPE) — **pas** un concept nommé de la littérature. Il compose des briques établies : RAG + PEFT (LoRA) + continual learning + distillation des logs.

Le RAG s'allège avec le temps

- Quand une connaissance se **stabilise**, elle est « graduée » dans le LoRA
- Moins de retrievals nécessaires → latence réduite
- La base vectorielle peut **shrink** : on retire ce qui est internalisé
- Le coût d'inférence diminue à chaque cycle

Le LoRA devient plus pertinent

- Entraîné sur des **vraies requêtes** (distribution réelle) — pas des hypothèses
- Réponses validées par les utilisateurs = **ground truth**
- Réduit le risque d'hallucination : le modèle apprend depuis le retrieval vérifié
- Chaque itération affine le style et le vocabulaire du domaine

1. Qualité des données — Le piège critique

- Si le RAG a retrouvé de mauvais chunks, et qu'on entraîne dessus → **on cuit les erreurs dans les poids**
- **Obligation** : pipeline de validation/filtrage avant chaque retraining
- Ne **jamais** distiller aveuglément les sorties du RAG

2. Catastrophic Forgetting

- Chaque retraining risque de dégrader les connaissances précédentes
- Solution : **replay buffer** — mélanger anciennes et nouvelles données
- Technique : EWC (Elastic Weight Consolidation)

3. Que "grader" du RAG au LoRA ?

- ✓ **Bons candidats** : requêtes haute fréquence, faits stables
- ✗ **Mauvais candidats** : prix, dates, tout ce qui est volatile
 - Règle : si ça change le mois prochain, ça reste dans le RAG

4. La cadence de retraining

- Pas de calendrier fixe — mesurer le **RAG hit rate** et la qualité du retrieval
- Certaines équipes : mensuel. D'autres : semestriel
- Retrainer quand les métriques l'indiquent, pas quand le calendrier le dit

Stack recommandée — Continual Learning Loop en Production

Entraînement LoRA

- **Axolotl / Unsloth** : frameworks de référence pour LoRA fine-tuning
- **HuggingFace PEFT** : contrôle fin sur le merge d'adapters
- **Weights & Biases** : tracking d'expériences — comparer cycle N vs $N-1$
- **GPU on-demand** : Modal, RunPod, Lambda Labs — pas de GPU permanent

RAG en production

- **Qdrant / pgvector** : vector DB (Weaviate si multi-tenant)
- **LlamaIndex** : pipelines RAG production-grade
- **vLLM / TGI** : serving avec hot-swap d'adapters LoRA
- **Langfuse** : observabilité — logs queries, chunks, feedback

Curation des données (la pièce critique)

- **Great Expectations / dbt** : valider et nettoyer les logs récoltés
- Scripts de classification : « connaissance stable » (LoRA) vs « volatile » (RAG)
- **Replay buffer** (S3) : anciennes données mélangées à chaque cycle

Évaluation avant chaque deploy

- **RAGAS** : faithfulness, relevancy, context precision
- **LLM-as-judge** : modèle plus fort (GPT-4, Claude) évalue sur un golden set maintenu entre cycles
- **Règle absolue** : ne jamais déployer un nouvel adapter sans comparer aux scores du cycle précédent. Régression → abort.

Travaux Pratiques

Objectifs pédagogiques

- Comprendre le pipeline complet SFT avec LoRA/QLoRA
- Manipuler la librairie peft et trl
- Mesurer l'impact du rang r et de α sur la qualité
- Comparer les résultats avec et sans LoRA

Dataset & Stack

- **Modèle** : Qwen/Qwen2.5-1.5B ou unsloth/Llama-3.2-1B
- **Librairies** : transformers, peft, trl, bitsandbytes
- **Dataset** : tatsu-lab/alpaca ou custom
- **Plateforme** : Google Colab (T4/L4 GPU)

Étapes du TP

1. **Chargement** : modèle en 4-bit avec `BitsAndBytesConfig`
2. **Configuration LoRA** : `LoraConfig(r=16,  $\alpha$ =32, target_modules=...)`
3. **Dataset SFT** : formater en paires instruction/réponse
4. **Entraînement** : `SFTTrainer` avec 3 epochs
5. **Inférence** : comparer base vs LoRA sur des prompts tests
6. **Bonus** : fusionner les poids (`merge_and_unload`)

Questions de réflexion

- Quel impact du rang r (4 vs 16 vs 64) sur la qualité et la vitesse ?
- Que se passe-t-il si on cible seulement Q/K vs Q/K/V/FFN ?
- Combien de Mo fait l'adapter LoRA sauvegardé ?
- Pourquoi le modèle de base répond-il mal aux instructions ?

Objectifs pédagogiques

- Implémenter un pipeline RAG complet (indexation + retrieval + génération)
- Comprendre l'impact du chunking sur la qualité des réponses
- Observer la différence Top-K vs MMR
- Débugger un pipeline RAG défaillant

Stack technique

- **Embedding** : paraphrase-multilingual-MiniLM
- **Vector Store** : ChromaDB (local, simple)
- **LLM** : llama-3.3-70b via Groq API
- **Framework** : LangChain
- **Corpus** : Articles Wikipedia en français

Étapes du TP

1. **Chargement** : WikipediaLoader (3 articles FR)
2. **Chunking** : tester 500 / 800 / 1500 tokens, visualiser l'overlap
3. **Indexation** : embedding + stockage ChromaDB
4. **Retrieval** : Top-K vs MMR, afficher les chunks
5. **Génération** : prompt template + LLM, RAG vs sans RAG
6. **Debug** : diagnostiquer un RAG qui dit "je ne sais pas"
7. **Bonus** : Hybrid Search BM25 + dense

Exercice de debug

Le système répond toujours "Je ne dispose pas de cette information".

Diagnostiquez : le problème vient-il du *retrieval* (mauvais chunks ou mauvais embeddings), du *prompt* (trop restrictif), ou du *LLM* (mauvaise config) ?

Ce que vous savez maintenant

- **Fine-tuning LLM** : SFT, DPO/RLHF, pipeline complet
- **LoRA & QLoRA** : $\Delta W = BA$, rang faible r , 99% moins de params
- **Familles LoRA** : DoRA, VeRA, LoRA+, AdaLoRA, LoKr
- **RAG** : $p(y|x) = \sum_z p(z|x) p(y|x, z)$, pipeline complet
- **Familles RAG** : Naive $\rightarrow$ Advanced $\rightarrow$ Modular $\rightarrow$ Agentic $\rightarrow$ Graph
- **Quand utiliser quoi** : LoRA + RAG = combinaison optimale

Aperçu de la Séance 7

Deep Learning pour la Vision Avancée

- Modèles Vision-Language : CLIP, ALIGN
- Génération d'images : Diffusion Models (DDPM, Stable Diffusion)
- Segmentation sémantique : SAM
- Vision Transformers en production

Questions ?